LANGCHAIN SERIES

Guardrails

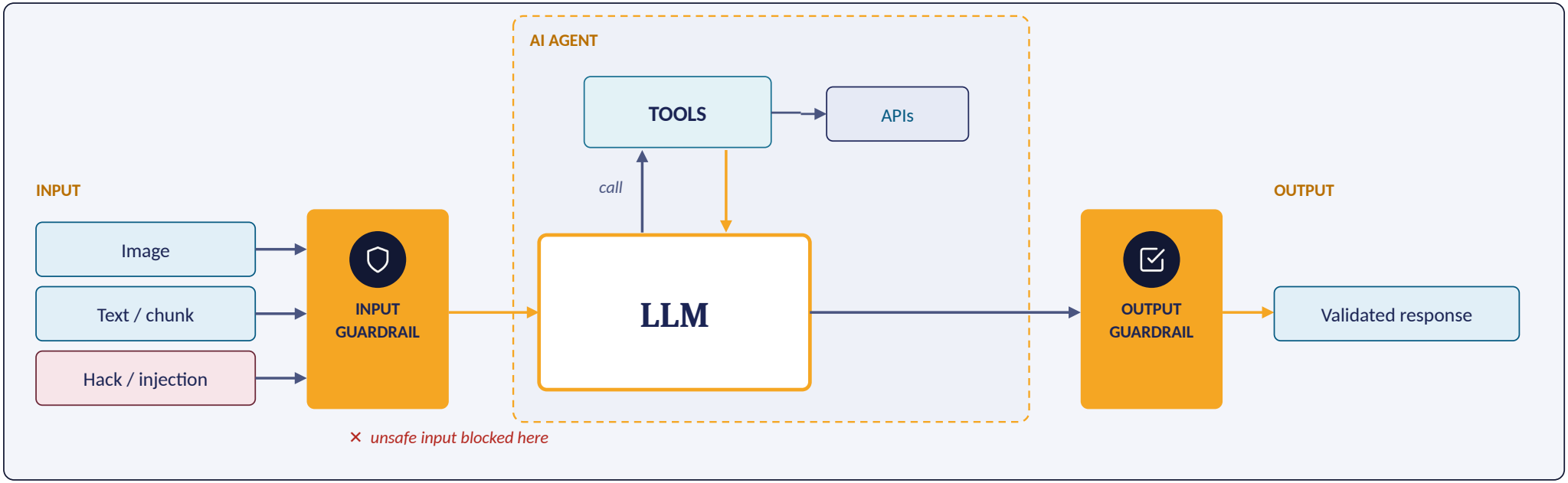
Safety checks and content filtering for agents

Validate and filter content at key points in your agent's execution — built on the LangChain middleware system.



What Are Guardrails?

Safety mechanisms that control what goes into and comes out of an AI agent — they sit around the whole agent pipeline.



Only safe inputs

Processes appropriate input

Only approved actions

Performs only the tool calls that passed the approval policy.

Only compliant outputs

Returns validated, policy-compliant responses to the user.

What Guardrails Protect Against

The failure modes that guardrails exist to catch: leaked personal data, unsafe input and output, and risky actions.



PII Leakage

Stop personal data from reaching the model

EXAMPLE

an email or a card number



Prompt Injection

Detect and block attacks embedded in user input

EXAMPLE

“ignore your previous instructions and…”



Human in the loop for specific actions

Pause before the agent runs a critical action

EXAMPLE

before booking a room or sending an email



Harmful Content

Block inappropriate or unsafe requests and replies

EXAMPLE

“how do I make explosives?”

Two Approaches

Rules are cheap and predictable. Models understand meaning. Real systems use both.

Deterministic guardrails

Rule-based logic — regex patterns, keyword matching.

- ✓ **Fast**
- ✓ **Predictable**
- ✓ **Cost-effective**
- ✕ **May miss nuanced violations**

Model-based guardrails

LLMs or classifiers evaluate content with semantic understanding.

- ✓ **Catches subtle issues rules miss**
- ✓ **Understands context and intent**
- ✕ **Slower**
- ✕ **More expensive**

LangChain ships built-in guardrails (PII, human-in-the-loop)

CODE 01 TWO APPROACHES

Deterministic vs. model-based guardrails

Deterministic rules in code vs. an LLM that judges the content

- Deterministic: keywords or regex. Fast, predictable and free, but blind to meaning.
- Model-based: understands intent, but every check costs a model call. Real systems use both.

PYTHON

```python
# Deterministic: fixed rules in plain Python, no model call
def deterministic_guardrail(text: str) -> bool:
    banned_keywords = ["hack", "exploit", "malware", "bomb"]   # the rule list
    # True if any banned word appears in the text (case-insensitive)
    return any(kw in text.lower() for kw in banned_keywords)

# Model-based: ask an LLM to judge the meaning
def model_based_guardrail(text: str) -> str:
    prompt = f"""Is the following user input safe to process?
Reply with only 'SAFE' or 'UNSAFE'.

Input: {text}"""                          # one-word answer = easy to parse
    # temperature=0 on the model keeps the verdict consistent
    return model.invoke([{"role": "user", "content": prompt}]).content.strip()

# "How do I hack into a database?"  ->  BLOCKED  |  UNSAFE
# "Explain how malware spreads"     ->  BLOCKED  |  SAFE   (model reads intent)
```

Kinds of Guardrail Middleware in LangChain

Middleware provides a way to more tightly control what happens inside the agent.



PII detection

Finds personal data such as emails, card numbers and IP addresses, then redacts, masks, hashes or blocks it.



Human-in-the-loop

Pauses before a sensitive tool runs and waits for a person to approve or reject the call. Best for high-stakes actions.



Before agent guardrails

Checks the request once, at the start of each invoke, and can block it before any model call.



After agent guardrails

Checks the final answer once, before it reaches the user: safety, quality or policy.



Combine multiple guardrails

Stack several in middleware=[...]. They run in order, so each layer adds its own protection.

Built-in: PII Detection

PIIMiddleware finds personally identifiable information and handles it with one of four strategies.

redact

Replace with a type label

`[REDACTED_EMAIL]`

mask

Partially obscure the value

`****-****-****-1234`

hash

Replace with a deterministic hash

`a8f5f167...`

block

Raise an exception on detection

`Error thrown`

BUILT-IN PII TYPES

- email
- credit_card

CODE 02

BUILT-IN · PII DETECTION

Built-in PII detection with PIIMiddleware

One PIIMiddleware per PII type, each with a strategy: redact, mask, hash or block

- redact → [REDACTED_EMAIL] mask → ****-****-****-1234 hash → a8f5f167... block → raises an exception
- Built-in types: email, credit_card, ip, mac_address, url. For anything else, pass a regex detector.

PYTHON

```python
from langchain.agents import create_agent
from langchain.agents.middleware import PIIMiddleware

agent = create_agent(
    model=model,
    tools=[customer_lookup],
    middleware=[                          # one PIIMiddleware per PII type
        # email -> [REDACTED_EMAIL] before the model sees the message
        PIIMiddleware("email", strategy="redact", apply_to_input=True),
        # card number -> ****-****-****-5100 (keeps the last 4 digits)
        PIIMiddleware("credit_card", strategy="mask", apply_to_input=True),
        PIIMiddleware("api_key",
                      detector=r"sk-[a-zA-Z0-9]{32}",   # custom regex detector
                      strategy="block",                 # raise, don't answer
                      apply_to_input=True),             # check the user's input
    ],
)
# "Here is my key: sk-abc..." -> Detected 1 instance(s) of api_key in text content
```

CODE 03

BUILT-IN · HUMAN-IN-THE-LOOP

Human-in-the-loop approval

HumanInTheLoopMiddleware + a checkpointer, then resume with Command on the same thread_id

- interrupt_on: True = pause before this tool, False = run it automatically. The checkpointer saves the paused run.
- "approve" runs the tool; "reject" with a reason skips it. Same thread_id resumes; a new one starts fresh.

PYTHON

```python
hitl_agent = create_agent(
    model=model,
    tools=[search_web, send_email, delete_records],
    middleware=[HumanInTheLoopMiddleware(interrupt_on={
        "send_email": True,          # pause: needs a human decision
        "delete_records": True,      # pause: needs a human decision
        "search_web": False})],      # safe: runs without asking
    checkpointer=InMemorySaver(),    # saves the paused run so it can resume
)

config = {"configurable": {"thread_id": "session_001"}}   # identifies the run
hitl_agent.invoke({"messages": [{"role": "user",
    "content": "Send an email to team@company.com about the Q4 results"}]},
    config=config)                   # stops before send_email runs

# Same thread_id -> resumes the paused run with the human's decision
hitl_agent.invoke(Command(resume={"decisions": [{"type": "approve"}]}),
                  config=config)     # approve -> email is sent
# or: {"type": "reject", "reason": "Too risky, needs DBA review"}
```

CODE 04

CUSTOM · DEFINE AND INTEGRATE

Custom guardrails: define it, then plug it in

Subclass AgentMiddleware, then pass an instance in create_agent(middleware=[...])

- before_agent runs once at the start (input check); after_agent once on the final answer (output check).
- Pass an instance: MyGuardrail(). before_* hooks run top to bottom, after_* hooks bottom to top.

PYTHON

```python
from langchain.agents import create_agent
from langchain.agents.middleware import AgentMiddleware, AgentState

# 1) Define: subclass AgentMiddleware and implement the hooks you need
class MyGuardrail(AgentMiddleware):
    def before_agent(self, state: AgentState, runtime) -> dict | None:
        # runs once per invoke, before the model: check the user's request
        return None      # None = let it through; a dict = update the state

    def after_agent(self, state: AgentState, runtime) -> dict | None:
        # runs once at the end: state["messages"][-1] is the final answer
        return None

# 2) Integrate: pass an INSTANCE (note the ()) in middleware=[...]
agent = create_agent(
    model=model,
    tools=[search_tool],
    middleware=[MyGuardrail()],       # add more instances to stack them
)
```

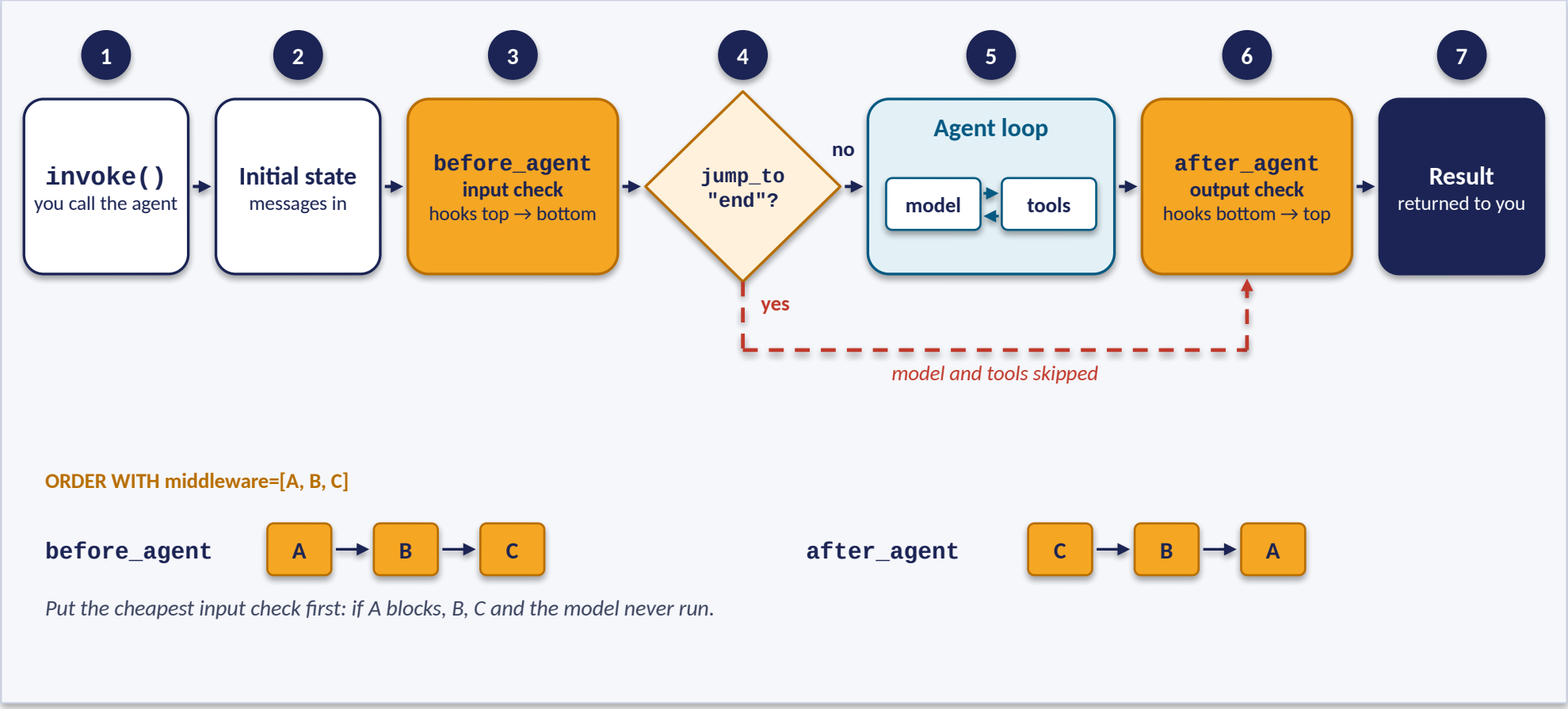
FLOW

CUSTOM · WHEN THE HOOKS RUN

One invoke, step by step

before_agent hooks guard the way in, after_agent hooks guard the way out

- Each hook runs once per invoke(), not once per model call. The model ⇄ tools loop in between can run many times.
- jump_to "end" skips later before_agent hooks, the model and the tools; after_agent still runs.



Put the cheapest input check first: if A blocks, B, C and the model never run.

CODE 05

CUSTOM · BEFORE-AGENT (INPUT)

Input guardrail with before_agent

Check the first human message; if it fails, return a reply and jump to the end

- A blocked request never reaches the model, so it costs zero LLM calls.
- "How do I hack into a server?" is answered by the filter's own message.

PYTHON

```python
class ContentFilterMiddleware(AgentMiddleware):
    def __init__(self, banned_keywords: list[str]):
        super().__init__()
        self.banned_keywords = [kw.lower() for kw in banned_keywords]  # lowercase once

    @hook_config(can_jump_to=["end"])           # allows exiting early
    def before_agent(self, state, runtime):      # runs before any LLM call
        first_message = state["messages"][0]     # the user's request
        if first_message.type != "human":
            return None                          # nothing to check
        content = first_message.content.lower()  # case-insensitive match
        for keyword in self.banned_keywords:
            if keyword in content:
                return {                         # replace the run with a refusal
                    "messages": [{"role": "assistant", "content":
                        "I cannot process requests containing inappropriate content. "
                        "Please rephrase your request."}],
                    "jump_to": "end",            # skip model and tools
                }
        return None                              # clean: continue normally

# Integrate: create_agent(..., middleware=[ContentFilterMiddleware(...)]), see 07
```

CODE 06

CUSTOM · BEFORE-AGENT IN A CHATBOT

before_agent in a multi-turn chatbot

[0] is the first message of the whole thread, so check the newest human message instead

- With a checkpointer, later turns are never checked, and a blocked first message blocks the thread for good.
- Don't scan the whole history either: one old bad message would block every turn.

PYTHON

```python
# Replaces before_agent inside ContentFilterMiddleware; integration is unchanged
@hook_config(can_jump_to=["end"])
def before_agent(self, state, runtime):
    # With a checkpointer, state["messages"] holds the WHOLE thread history.
    # Walk it backwards: the newest human message is this turn's request.
    last_human = next(
        (m for m in reversed(state["messages"]) if m.type == "human"), None
    )
    if last_human is None:
        return None                       # no user message: nothing to check

    content = last_human.content.lower()
    for keyword in self.banned_keywords:
        if keyword in content:
            return {                      # block only this turn
                "messages": [{"role": "assistant", "content":
                    "I cannot process requests containing inappropriate content."}],
                "jump_to": "end",
            }
    return None                           # the next turn is checked fresh
```

CODE 07

CUSTOM · EXITING EARLY

Exiting early with jump_to "end"

The decorator lets a hook exit early; returning jump_to "end" stops the run there

- The model and tools are skipped, and the hook's message becomes the reply.
- Watch out: in the notebook's healthcare bot, the after_agent disclaimer was still added to a blocked reply.

PYTHON

```python
@hook_config(can_jump_to=["end"])        # declare: this hook may exit early
def before_agent(self, state, runtime):  # ...your check goes here
    return {
        "messages": [{"role": "assistant", "content": "Request blocked."}],
        "jump_to": "end",                # skip model and tools
    }

# Create agent with content filter
filtered_agent = create_agent(
    model=model,
    tools=[search_tool],
    middleware=[ContentFilterMiddleware(
        banned_keywords=["hack", "exploit", "malware", "jailbreak", "bypass"])],
)

result = filtered_agent.invoke({"messages": [{"role": "user",
    "content": "How do I hack into a server?"}]})
print(result["messages"][-1].content)    # last message = the hook's reply
# I cannot process requests containing inappropriate content. ...
```

CODE 08

CUSTOM · AFTER-AGENT (OUTPUT)

Output guardrail with after_agent

In after_agent, send the final AIMessage to a judge model and replace it if flagged

- The notebook's judge flags any answer that names a competitor product.
- To replace the answer, return a message with the same id. (The notebook edits .content in place instead.)

PYTHON

```python
class SafetyGuardrailMiddleware(AgentMiddleware):

    def after_agent(self, state, runtime):         # runs on the final answer
        last_message = state["messages"][-1]
        if not isinstance(last_message, AIMessage):
            return None                            # only judge AI answers
        safety_prompt = f"""Reply with only 'SAFE' or 'UNSAFE'.
Mark UNSAFE if the response mentions any competitor product by name.

Response to evaluate:
{last_message.content}"""
        result = model.invoke([{"role": "user", "content": safety_prompt}])  # judge
        if "UNSAFE" not in result.content.upper():
            return None                            # safe: no change
        # Return the change. Same id -> replaces the answer instead of appending
        return {"messages": [AIMessage(content="I'm unable to provide that response.",
                                       id=last_message.id)]}

# Integrate: pass an instance in middleware=[...]
safe_agent = create_agent(model=model, tools=[general_tool],
                          middleware=[SafetyGuardrailMiddleware()])
```

CODE 09 LAYERED GUARDRAILS

Layered guardrails

List them in middleware=[...]: before_* hooks run top to bottom, after_* hooks bottom to top

- Put cheap rule-based input checks at the top: "how to make a malware?" stops at layer 1, so no other layer or model call runs.
- Output hooks (after_*) of the same kind run in reverse list order, so plan their order from the bottom up.

PYTHON

```python
production_agent = create_agent(
    model=model,
    tools=[search_tool, send_email_tool],
    middleware=[       # before_* hooks: top -> bottom | after_* hooks: bottom -> top
        # Layer 1: Deterministic input filter (custom, before_agent)
        ContentFilterMiddleware(banned_keywords=["hack", "exploit", "malware"]),
        # Layer 2: PII redaction on input
        PIIMiddleware("credit_card", strategy="mask", apply_to_input=True),
        # Layer 3: Human approval for sensitive tools
        HumanInTheLoopMiddleware(
            interrupt_on={"send_email_tool": True, "search_tool": False}),
        # Layer 4: PII redaction on output
        PIIMiddleware("email", strategy="redact", apply_to_output=True),
        # Layer 5: Model-based output safety (custom, after_agent)
        SafetyGuardrailMiddleware(),
    ],
    checkpointer=InMemorySaver(),    # needed by HumanInTheLoopMiddleware
)
```

CODE 10

USE CASE · HEALTHCARE CHATBOT

Healthcare chatbot: a disclaimer on every answer

An after_agent hook returns the answer with the disclaimer added, when it is missing

- Return None = no change. To change the answer, return a message with the same id.
- The notebook appends to .content in place instead: it works, but relies on a side effect.

PYTHON

```python
class MedicalOutputValidator(AgentMiddleware):
    """Ensure all responses include appropriate medical disclaimers."""

    DISCLAIMER = ("\n\n*This is general health information, not medical advice. "
                  "Please consult a qualified healthcare professional.*")

    def after_agent(self, state, runtime):       # runs on the final answer
        last_message = state["messages"][-1]
        if not isinstance(last_message, AIMessage):
            return None                          # nothing to change
        if "medical advice" in last_message.content.lower():
            return None                          # disclaimer already there
        # Return the change. Same id -> replaces the old message, not appended
        return {"messages": [AIMessage(content=last_message.content + self.DISCLAIMER,
                                       id=last_message.id)]}
```

CODE 11

USE CASE · HEALTHCARE CHATBOT

Healthcare chatbot: the full guardrail stack

Your custom guardrails go in middleware=[...] right next to the built-in ones

- HealthcareSafetyFilter (custom before_agent) blocks off-topic or harmful requests before any LLM call.
- MedicalOutputValidator (custom after_agent) adds the disclaimer to every reply, including blocked ones.

PYTHON

```python
healthcare_bot = create_agent(
    model=model,
    tools=[search_symptoms, book_appointment, get_medication_info],
    middleware=[
        HealthcareSafetyFilter(),              # custom: blocks harmful topics
        PIIMiddleware("email", strategy="redact", apply_to_input=True),
        PIIMiddleware("credit_card", strategy="mask", apply_to_input=True),
        HumanInTheLoopMiddleware(interrupt_on={
            "book_appointment": True,          # booking needs approval
            "search_symptoms": False,
            "get_medication_info": False,
        }),
        MedicalOutputValidator(),              # custom: adds the disclaimer
    ],
    checkpointer=InMemorySaver(),              # needed for approval + chat history
    system_prompt="You are a helpful healthcare assistant. ...",
)
# "how to hack ? a hospital"         -> blocked by HealthcareSafetyFilter
# "Book an appointment for Kerolos"  -> pauses for human approval
```